

Questions for Hackathon

1. Design and verify a comprehensive scan chain test circuit incorporating normal flip-flops (FF), scan flip-flops (SFF), scan stitching, false paths, and multicycle paths. Use only Cadence Genus for synthesis/timing analysis and RTL Simulation for functional verification. Target a 90nm library (use available .lib/.lef files in lab). Complete the following formal requirements:

Part A: Circuit Architecture (RTL Design)

Implement the Verilog module ``scan_test_core`` with:

- 16 normal DFFs (``normal_ff[15:0]``) in a counter datapath (clk, rst, enable).
- 16 scan DFFs (``scan_ff[15:0]``) with scan port architecture: each SFF has D (functional), SI (scan_in), SE (scan_enable), CLK, RST → Q.
- Scan chain stitching: connect all 16 SFFs into one continuous scan chain (scan_in_global → SFF0.SI → SFF1.SI → ... → SFF15.SI → scan_out_global).
- False path control: 4:1 MUX selecting counter output (from `normal_ff[7:0]`) with select ``false_path_en`` (when asserted, bypasses logic but STA must ignore this path).
- Multicycle path data path: 8-bit pipeline with 3-cycle multicycle path between ``mc_launch_ff[7:0]`` → complex combo logic (add+multiply) → ``mc_capture_ff[7:0]``.
- Control signals: ``clk`` (400 MHz target), ``rst_n``, ``scan_en``, ``false_path_en``, ``mc_en``, ``test_mode``.

Part B: Synthesis & Timing Analysis (Cadence Genus)

1. Synthesize using Genus: ``read_hdl scan_test_core.v``, ``elaborate``, target 400 MHz.
2. SDC constraints (create ``scan_test.sdc``):

```

```
create_clock -name clk -period 2.5 [get_ports clk]

set_input_delay 0.5 -clock clk [all_inputs]

set_output_delay 0.5 -clock clk [all_outputs]

Multicycle path (3-cycle):

set_multicycle_path -setup 3 -from [get_pins mc_launch_ff[*]/Q] -to [get_pins mc_capture_ff[*]/D]

set_multicycle_path -hold 2 -from [get_pins mc_launch_ff[*]/Q] -to [get_pins mc_capture_ff[*]/D]

False path:
```

```
set_false_path -from [get_ports false_path_en] -to [get_pins false_path_mux/Z]
```

```
...
```

### 3. Generate reports:

- Scan stitching overhead: ``report_area`` pre/post-scan insertion; quantify wiring + mux overhead (~15% target).
- Show scan chain connectivity (``report_scan_def`` or equivalent).
- Timing closure: ``report_timing`` -show all critical paths. Verify MCP slack >0.2ns, false path excluded from analysis.
- PPA trade-off: ``report_power`` (dynamic + leakage); compare scan vs. non-scan versions.

## Part C: RTL Verification

Create self-checking testbench ``tb_scan_test_core.v``:

1. Functional mode: Reset → enable counter → verify `normal_ff` increments.
2. Scan shift mode: ``scan_en=1``, shift known pattern (0xAA55) through full 16-SFF chain → capture ``scan_out_global`` → verify pattern rotates correctly.
3. False path test: Toggle ``false_path_en`` → confirm counter output ignored (monitor mux output unchanged).
4. Multicycle verification: Apply 3-cycle stimulus → confirm ``mc_capture_ff`` matches delayed computation.
5. Coverage: Achieve >95% toggle + line coverage; include corner cases (rst during scan, async `false_path_en`).

## Deliverables

```
scan_test_core.v # Main RTL
tb_scan_test_core.v # Testbench
scan_test.sdc # Constraints
genus_report_timing.rpt # Post-synth timing (slack >0.1ns all paths)
genus_report_area.rpt # Scan overhead quantification
genus_report_power.rpt # Dynamic/leakage breakdown
genus_scan_def.scandef # Chain definition
RTL_Simulation_waves.vcd # Key waveforms (scan shift, MCP)
RTL_Simulation_coverage.rpt # >95% coverage
README.md # Explain design choices, STA fixes applied
```

2. **Design Objective:** Implement, synthesize, and verify three 8-bit multiplier architectures—Array, Wallace Tree (WT), and Dadda—using Cadence Genus (90nm library) and RTL Simulation. Perform comprehensive PPA comparison targeting 800 MHz operation.

### Part A: RTL Architecture Requirements

Create Verilog module `multiplier_compare` with three independent 8x8 unsigned multipliers:

- Array Multiplier (`array_mult`): Regular 8x8 AND array + 8-stage ripple-carry adders for PP reduction
- Wallace Tree Multiplier (`wt_mult`): 3-stage reduction using 3:2, 4:2 compressors → final 8-bit CPA
- Dadda Multiplier (`dadda_mult`): Optimal-height reduction (heights: 15→9→5→3→2→1) using compressor matrix

Compressor Requirements:

```
module compressor_3to2 (A,B,C, Sum, Carry); // Full adder
```

```
module compressor_4to2 (A,B,C,D, Sum, Carry1, Carry2); // Half+full adders
```

### Part B: Cadence Genus Synthesis Flow

# Synthesis script: `synth_multiplier.tcl`

```
read_hdl multiplier_compare.v
```

```
elaborate
```

# Common constraints

```
create_clock -name clk -period 1.25 [get_ports clk]
```

```
set_max_area 0
```

```
set_max_fanout 8 [all_outputs]
```

```
Multi-corner timing
```

```
read_lib 28nm_typ.lib; read_lib 28nm_slow.lib; read_lib 28nm_fast.lib
```

```
compile -timing_high_effort_script
```

Required Genus Reports (submit all):

- `report_timing -significant_digits 3 > timing_all.rpt` (slack >0.15ns critical path)
- `report_area -hierarchy > area_breakdown.rpt` (quantify compressor vs. adder area)
- `report_power -hierarchy -verbose > power_analysis.rpt` (dynamic/leakage power at 800MHz)

PPA Table: Create markdown table comparing all three architectures:

- Cell Area ( $\mu\text{m}^2$ )
- Leakage power ( $\mu\text{W}$ )
- Dynamic Power (mW)
- Worst Neg Slack (ns) Freq (MHz)

### Part C: RTL Simulation Verification Suite

Self-checking testbench `tb_multiplier_compare.v`:

- Accuracy Test:  $10^5$  random input pairs  $\rightarrow$  verify  $|\text{output\_error}| < 0.01\%$
- Stress Patterns: All-ones, alternating 1s/0s, MSB-only  $\rightarrow$  check compressor carry propagation
- Coverage:  $>98\%$  toggle coverage on all compressor bits

### Deliverables:

```
multiplier_compare.v # Complete RTL w/ compressors
tb_multiplier_compare.v # Self-checking TB
synth_multiplier.tcl # Genus script
timing_all.rpt # All paths slack $>0.15\text{ns}$
area_breakdown.rpt # Architecture hierarchy
power_analysis.rpt # Dynamic/leakage breakdown
ppa_comparison.md # Formatted table + analysis
RTL_Simulation_coverage.rpt # $>98\%$ toggle coverage
multiplier_waves.vcd # PP reduction stages
```

### 3. Design a unified memory controller supporting SRAM (single-port), DRAM (with refresh), and Cache controller (4KB, 4-way set-associative).

#### Part A: Architecture Requirements

```
module memory_controller (
 clk, rst_n,
 // SRAM port: addr[15:0], din[31:0], dout[31:0], we, en
 // DRAM port: cmd, addr[12:0], bank[1:0], data[31:0], refresh_req
 // Cache port: cpu_addr[11:0], cpu_din, cpu_dout, cpu_we, hit, dirty
);
```

SRAM Controller: Address decode + read/write arbitration

DRAM Controller: Auto-refresh every 8K cycles, PRECHARGE-ACTIVATE-RD/WR bursts

Cache Controller: LRU replacement, write-back policy, 64B lines (16 sets × 4 ways)

## Part B: Genus Timing/Power Analysis

### # Multi-port timing constraints

```
set_multicycle_path -setup 2 -from dram_refresh_ff -to dram_cmd_ff
```

```
set_false_path -from cpu_addr[11] -to lru_update_logic # Tag comparison false path
```

## Part C: RTL Simulation Test Plan

- SRAM: 1000 rand R/W → data integrity
- DRAM: Refresh collision handling → no data loss
- Cache: Compulsory/conflict/capacity misses → 95% hit rate on LRU traces
- Mixed-mode: CPU access during DRAM refresh → priority arbitration

### Deliverables:

- memory\_controller.v # Main RTL with SRAM/DRAM/Cache unified ports
- tb\_memory\_controller.v # Self-checking testbench (rand R/W, refresh stress)
- synth\_memory.tcl # Genus synthesis script w/ MMMC views
- memory\_controller.sdc # SDC w/ multicycle refresh paths, false path on tag\_cmp
- timing\_report.rpt # report\_timing -path\_full -slack\_lesser\_than 0.2
- area\_hierarchy.rpt # report\_area -hierarchy (controller vs. cache array)
- power\_dynamic\_ir.rpt # report\_power w/ VCD activity, IR drop analysis
- cache\_hit\_rate.txt # RTL Simulation log: 95% hit rate on 10k access trace
- RTL\_Simulation\_coverage.rpt # >97% line/toggle coverage
- memory\_waves.vcd # DRAM refresh burst + cache miss waveforms
- ppa\_tradeoffs.md # Analysis: SRAM speed vs. DRAM density, cache LRU overhead

## 4. Design a 32-bit Universal Arithmetic Unit with configurable counter, barrel shifter, and 5-stage ALU.

### Part A: Architecture

```
module universal_alu (
 clk, rst_n,
 op_sel[3:0], // ADD=0, SUB=1, SHL=2, SHR=3, ROL=4, CNT=5
 data_in[31:0], enable,
 data_out[31:0], valid
```

);

Counter Mode: Up/down, loadable, glitch-free gating

Shifter: 32-bit barrel (configurable rotate/shift direction)

ALU: Booth multiply + leading zero count + population count

## Part B: Genus Optimization

Target 1.2 GHz, multi-Vt optimization:

```
set_attribute [get_cells alu_crit_path] hvt false
```

```
size_cell -high_effort_timing [get_cells shifter_*]
```

## Part C: Verification

Sequence: Counter inc→ALU add→shift→multiply pipeline

Coverage: All opcodes + boundary conditions (all-zero, all-one)

### Deliverables:

- universal\_alu.v # RTL w/ op\_sel muxing counter/shifter/ALU
- tb\_universal\_alu.v # Pipeline test: cnt→add→shift→popcnt sequence
- synth\_alu.tcl # Genus script w/ multi-Vt, high\_effort compile
- universal\_alu.sdc # Clock uncertainty 20ps, I/O delays 0.4ns
- timing\_critical.rpt # report\_timing -critical\_paths (Booth mult slack)
- area\_alu\_breakdown.rpt # Counter 15%, shifter 25%, ALU 60% area split
- power\_pareto.rpt # report\_power @1.2GHz,  $\alpha=0.3$  (glitch-free gating verify)
- opcode\_coverage.txt # RTL Simulation: All 6 ops exercised, popcnt accuracy 100%
- RTL\_Simulation\_waves.vcd # Barrel shift rotate + counter wraparound
- coverage\_report.rpt # >98% toggle on datapath bits
- alu\_ppa\_analysis.md # LVT crit path speedup vs. HVT leakage savings
- README\_alu.md # Glitch mitigation in counter, barrel logN depth proof

## 5. Design a dual-mode FSM controller for H.264 CABAC decoder with safe state recovery and multicycle paths.

### Part A: Architecture

- Mode 0: Sequence detector (any 1011 pattern)
- Mode 1: Cache coherence controller (MESI protocol)
- One-hot encoding, gray code transitions, power-optimized gray-to-binary decoder

### Part B: Genus Constraints

```
set_case_analysis 0 [get_pins reset_asserted]
```

set\_multicycle\_path -setup 4 -from ctx\_model\_update -to bin\_idx\_ff

### Part C: Test Cases

- FSM coverage >100% state/transition
- Reset during transition → safe state
- Illegal input sequences → recovery state

### Deliverables:

- fsm\_dual\_mode.v # One-hot MESI + seq detector, gray transitions
- tb\_fsm\_controller.v # H.264-like traces + illegal seq recovery
- synth\_fsm.tcl # Genus w/ set\_case\_analysis on reset
- fsm\_controller.sdc # 4-cycle MCP on ctx\_update, false\_path on unused states
- timing\_state\_trans.rpt # report\_timing -from/to state\_regs (all transitions)
- area\_encoding.rpt # One-hot vs. binary area comparison
- power\_state\_hold.rpt # Leakage min in idle states
- state\_coverage.txt # RTL Simulation: 100% state/transition, MTBF calc log
- fsm\_waves.vcd # Reset mid-transition recovery waveform
- RTL\_Simulation\_assertions.log # Assertion pass: no illegal states held >2 cycles
- fsm\_analysis.md # Gray code reduces toggle power by 50%
- README\_fsm.md # Safe recovery FSM, CABAC ctx model integration notes
- genus\_dft\_check.rpt # No combinational loops detected

- 6. Design, synthesize, and comprehensively characterize five distinct 32-bit adder architectures to evaluate trade-offs in performance, power, and area (PPA) metrics using Cadence Genus synthesis (90nm PDK) and RTL verification. This benchmark evaluates mastery of parallel prefix adder theory, carry propagation optimization, and production-level STA flows.**

### Part A: Architecture Specifications

Implement the Verilog module `adder_benchmark_suite` instantiating five independent 32-bit unsigned adders with identical interfaces:

```
verilog

module adder_benchmark_suite (
 input clk, rst_n,
 input [31:0] a_in, b_in, cin,
 input [4:0] arch_sel, // 0=RCA, 1=CSKA, 2=CLA, 3=CSA, 4=KSA
 output reg [32:0] sum_out,
 output reg [4:0] valid_arch
```

);

Required Implementations (each with hierarchical/modular structure):

- Ripple Carry Adder (RCA) - Baseline (1-bit FA ripple chain)
- Carry Skip Adder (CSKA) - Variable skip blocks (4-8-16-4 bit groups, overlapping)
- Carry Lookahead Adder (CLA) - 4-way hierarchical (4-bit blocks → 16-bit → 32-bit)
- Carry Save Adder (CSA) + CPA - 3-stage CSA reduction → final 32-bit ripple CPA
- Kogge-Stone Adder (KSA) - Parallel prefix with optimal fanout=2 topology

## Part B: Cadence Genus Synthesis & STA Flow

Synthesis Script Requirements (synth\_adders.tcl):

```
Multi-corner MMMC setup

read_mmmc adders_view.mmmc

read_hdl adder_benchmark_suite.v

elaborate -root adder_benchmark_suite

Aggressive timing constraints

create_clock -name clk -period 1.0 -waveform {0 0.5} [get_ports clk]

set_input_delay 0.2 -clock clk [get_ports {a_in[*] b_in[*] cin}]

set_output_delay 0.3 -clock clk [get_ports {sum_out[*]}]

set_max_transition 80ps [all_outputs]

set_max_capacitance 50fF [all_outputs]

set_max_fanout 10 [all_outputs]

Multi-Vt optimization

compile_ultra -timing_high_effort_script -incremental

DFT preparation

check_dft_rules -scan -verbose
```

Mandatory Genus Commands & Reports:

- Timing Analysis: report\_timing -path\_type full\_clock -max\_paths 50 - slack\_lesser\_than 0.1 > timing\_critical.rpt
- Area Hierarchy: report\_area -hierarchy -nosplit > area\_detailed.rpt
- Power Profiling: report\_power -hierarchy -connectivity > power\_full.rpt
- Scan Chain Prep: preview\_dft -scan -full (quantify insertion overhead)

## Part C: Quantitative PPA Benchmark Matrix



Submit formatted table ppa\_analysis.md with exact Genus-extracted values by comparing the following parameters for all adder architectures:

- Cell Area (GE)
- Cell Area ( $\mu\text{m}^2$ )
- Worst Neg Slack (ps)
- Total Dynamic Power ( $\mu\text{W}$  @1GHz)
- Leakage Power (nW)
- Max Fanout
- Latency (cycles)
- Power-Delay Product (fJ)

### Part D: RTL Simulation Golden Reference Verification

Self-checking testbench tb\_adder\_benchmark.v MUST demonstrate:

- Functional Accuracy: Random input pairs per architecture  $\rightarrow$  100% bit-match
- Carry Propagation Stress:
  - Pattern 1:  $0x00000000 + 0xFFFFFFFF$  = carry ripple
  - Pattern 2:  $0x55555555 + 0xAAAAAAAA$  = alternating carry
  - Pattern 3:  $0x80000000 + 0x80000000$  = MSB overflow
- Timing Corner Cases: Clock stretched 20%  $\rightarrow$  no metastability

### Deliverables

- adder\_benchmark\_suite.v # Complete 5-architecture RTL
- tb\_adder\_benchmark.v # Self-checking + stress patterns
- synth\_adders.tcl # Production Genus flow
- adders.mmmc # Multi-corner setup
- adders.sdc # Timing constraint suite
- timing\_critical.rpt # WNS  $>0.1\text{ns}$  target
- area\_detailed.rpt # Hierarchical breakdown
- power\_full.rpt # Dynamic/leakage @1GHz  $\alpha=0.25$
- dft\_preview.rpt # Scan chain readiness
- RTL\_Simulation\_coverage.rpt #  $>98\%$  toggle on carry logic
- adder\_latency.txt | Arch | Latency |
- waves\_carry\_propagation.vcd # Visual carry wavefront comparison
- ppa\_analysis.md # Formatted table + winner analysis
- design\_justification.pdf # CSKA math, KSA optimality proof